

Project 1 Documentation

Project: G12_Project1_xv6CustomizeSystemCalls

Scope: xv6-riscv extension with custom system calls, user-facing kernel services, and dedicated user-space tests.

1. Overview

Project 1 extends xv6 in three main areas: inter-process communication, process management, and memory management. The implementation adds new system calls, registers them in the syscall table, exposes wrappers in user space, and includes test programs that run from the xv6 shell.

The tree also changes two existing interfaces. The exposed sleep call is `pause(int ticks)`, and the memory-growth path supports eager and lazy allocation modes through the `sbrk` wrapper layer.

2. Features

- Shared memory: `shmget`, `shmat`, `shmdt`, `shmctl`
- User-visible locks: `lockinit`, `lockacquire`, `lockrelease`, `locktry`, `lockcheck`
- Message queue IPC: `sendmsg`, `recvmsg`, `broadcast`
- Process utilities: `getprocsinfo`, `getppid`, `sleep2`
- Child limit control: `setchildlimit`
- Signal registration: `signal`
- Memory interface changes: `pause` plus eager/lazy `sbrk` support

3. Implementation

3.1 Syscall integration

The syscall path follows normal xv6 structure. `kernel/syscall.h` assigns syscall numbers, `kernel/syscall.c` registers handlers, `user/user.h` exposes declarations, and `user/usys.pl` generates the trap stubs used by user programs.

3.2 Shared memory

Shared memory support is implemented in `kernel/sysproc.c` with supporting process metadata in `kernel/proc.h` and declarations in `kernel/defs.h`. The interface supports segment creation, attachment, detachment, and deletion.

The `shmtest` program validates the full lifecycle by creating a segment, attaching it, writing to it, detaching it, and deleting it.

3.3 User lock table

The lock subsystem exposes a small lock table to user processes. A caller can initialize a lock identifier, acquire it, attempt a non-blocking acquire, release it, and query whether the lock is currently held.

`locktest` exercises success paths, busy-state reporting, and re-acquisition after release.

3.4 Message passing

Message queue support is implemented in `kernel/sysmsg.c`. The interface provides direct parent-to-child style delivery through `sendmsg` and `recvmsg`, plus multi-recipient delivery through `broadcast`.

`msgtest` checks both direct delivery and broadcast delivery to multiple child processes.

3.5 Process utilities

`getppid` returns the parent PID, `sleep2` provides alternate kernel tick-based sleeping, and `getprocsinfo` exposes process state information for inspection or debugging.

proctest validates these utilities together by checking parent identity, measuring sleep duration, and invoking the process-info path.

3.6 Child limit enforcement

setchildlimit adds a per-process cap on live children. The kernel checks this limit on the process creation path rather than introducing a separate process-management subsystem.

childlimittest and sclimit cover denial when the limit is reached and normal recovery after children exit and are reaped.

3.7 Signal registration

The signal syscall stores a handler pointer in process state. The current tree supports registration only. It does not implement full asynchronous signal delivery or automatic handler invocation.

signaltest confirms registration success and documents the implementation boundary.

3.8 Memory management changes

The user library routes sbrk through sys_sbrk with an allocation mode so the tree can expose both eager allocation and lazy allocation through sbrklazy. usertests contains additional coverage for these paths.

4. Testing

- shmtest: shared memory create, attach, write, detach, delete
- locktest: lock initialization, acquire, busy detection, release, retry
- msgtest: direct message delivery and broadcast delivery
- proctest: getppid, sleep2, getprocsinfo
- childlimittest and sclimit: child-limit enforcement and boundary behavior
- signaltest: signal registration path
- usertests: broader regression coverage, including pause and memory-growth behavior

5. Build and execution

```
cd G12_Project1_xv6CustomizeSystemCalls
make qemu
```

```
Inside xv6:
$ shmtest
$ locktest
$ msgtest
$ proctest
$ childlimittest
$ sclimit
$ signaltest
```

6. Deliverables

- Kernel-side custom syscall implementations
- User-space syscall wrappers and trap stubs
- Feature-specific test programs included in fs.img
- Modified process and memory-management behavior required by the assignment

Generated from the current repository state on 2026-04-14.